**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ**

**(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

# ОТЧЕТ

по лабораторной работе № 2
по дисциплине «Операционные системы»

## Тема: Управление процессами и межпроцессное взаимодействие (Pipes) в POSIX

**Выполнил:**
студент группы М8О-203БВ-25
Гаврилов Данила Александрович

**Проверил:**
преподаватель
Ядров Артём

Москва – 2026

# 1 Цель работы

Практическое освоение системных вызовов операционных систем семейства *nix для управления процессами (`fork`, `exec`, `waitpid`) и организации межпроцессного взаимодействия с использованием неименованных каналов (`pipe`).

# 2 Краткое описание архитектуры решения

В соответствии с требованиями к модульности, проект разделен на три логических блока:

- **Заголовочный файл (Интерфейс):** Описывает платформонезависимые структуры и функции-обертки (`process_wrapper.h`). Скрывает детали реализации POSIX API.
- **Слой POSIX (Реализация):** Инкапсулирует системные вызовы ОС (`pipe`, `fork`, `dup2`, `execlp`). Включает обязательную обработку ошибок каждого системного вызова с возвратом статуса.
- **Вычислительный модуль и точка входа:**
  - Процесс-родитель считывает пользовательский ввод и передает данные через пайп.
  - Процесс-ребенок (запускаемый через `execlp`) перехватывает `stdin` через `dup2`, считывает строки, производит вычисления (деление чисел) и записывает результаты в файл. Защищен от `SIGFPE` при делении на ноль.

Для предотвращения состояния «зомби» (Zombie Process) родительский процесс использует блокирующий системный вызов `waitpid` с флагом `0` перед полным завершением работы.

# 3 Листинг ключевых фрагментов кода

Ниже представлены исключительно архитектурно значимые фрагменты кода, демонстрирующие работу с системным API, перенаправление файловых дескрипторов и синхронизацию.

## 3.1 Реализация системных вызовов (POSIX API)

```
bool create_child(const char* program_file, const char* out_file, pipe_t& p, process_t&
    child_process) {
    int f = fork();
    if(f == -1) {
        return false;
    }
    if(f > 0) {
        child_process.current_process = f;
        return true;
    }
    if(f == 0) {
        dup2(p.read_fd, 0);
        close(p.read_fd);
        close(p.write_fd);

        execlp(program_file, program_file, out_file, nullptr);

```

```
        _exit(1);
    }
}
```

Listing 1: Создание процесса и перенаправление дескрипторов

```
bool is_process_alive(process_t& pr) {
    int status;
    int w = waitpid(pr.current_process, &status, WNOHANG);
    if(w == 0) return true;
    return false;
}

int wait_child_end(process_t& child) {
    int status;
    return waitpid(child.current_process, &status, 0);
}
```

Listing 2: Синхронизация и предотвращение зомби-процессов

## 4 Результаты работы

Демонстрация корректного выполнения программы: успешное вычисление для первой строки и безопасное штатное завершение работы при обнаружении краевого случая (деления на ноль) во второй строке.

```
$ ./parent
tesr.txt
100 2 5
50 0
$ cat tesr.txt
10
```

## 5 Выводы

В ходе выполнения лабораторной работы была спроектирована и реализована многопроцессная архитектура в ОС Linux. Успешно решены следующие инженерные задачи:

- Отработано использование системного вызова `pipe` для создания однонаправленного канала связи.
- Произведена подмена файловых дескрипторов с помощью `dup2`, что позволило дочернему процессу прозрачно считывать данные стандартным потоком `std::cin`.
- Реализовано строгое управление ресурсами ОС: закрытие неиспользуемых концов трубы в каждом процессе предотвращает утечки файловых дескрипторов и обеспечивает корректную отправку `EOF`.
- Разделены сценарии применения системного вызова `waitpid`. Применен неблокирующий опрос (`WNOHANG`) для асинхронного мониторинга и блокирующее ожидание для корректной сборки статусов завершения ядром ОС, что полностью исключает утечку ресурсов через процессы-сироты или зомби.
- Реализована обработка кодов возврата системных вызовов и корректная обработка краевого случая (деление на ноль) без генерации аппаратного исключения `SIGFPE`.